Pandas — LEVEL 10: Sorting

Line-by-line code, output, examples, explanations, and easy interview points.

1. sort_values()

Code

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John", "Neha"],
    "Age": [22, 30, 25, 20],
    "Salary": [30000, 60000, 45000, 25000]
})

result = df.sort_values("Age")

print(result)
```

Output

```
   Name  Age  Salary
3  Neha   20   25000
0  Amit   22   30000
2  John   25   45000
1  Riya   30   60000
```

Explanation: sort_values() sorts rows using values in one or more columns.

Easy interview explanation: Use sort_values() when you want to arrange a DataFrame by column values.

2. Ascending Sorting

Code

```
result = df.sort_values("Salary", ascending=True)

print(result)
```

Output

```
   Name  Age  Salary
3  Neha   20   25000
0  Amit   22   30000
2  John   25   45000
1  Riya   30   60000
```

Explanation: ascending=True sorts from the smallest value to the largest.

Easy interview explanation: Ascending is the default sorting direction.

3. Descending Sorting

Code

```
result = df.sort_values("Salary", ascending=False)

print(result)
```

Output

```
   Name  Age  Salary
1  Riya   30   60000
2  John   25   45000
0  Amit   22   30000
```

```
3  Neha   20   25000
```

Explanation: ascending=False sorts from largest to smallest.

Easy interview explanation: Use descending sorting when you want the highest values first.

4. Sort Multiple Columns

Code

```
df = pd.DataFrame({
    "Department": ["IT", "HR", "IT", "HR"],
    "Salary": [50000, 40000, 50000, 30000],
    "Name": ["Amit", "Riya", "John", "Neha"]
})

result = df.sort_values(
    ["Department", "Salary"],
    ascending=[True, False]
)

print(result)
```

Output

```
  Department  Salary  Name
1          HR   40000  Riya
3          HR   30000  Neha
0          IT   50000  Amit
2          IT   50000  John
```

Explanation: The first column is the primary sort and the second column breaks ties.

Easy interview explanation: ascending can be a list, so each column can have its own direction.

5. sort_index()

Code

```
df = pd.DataFrame(
    {"Name": ["Amit", "Riya", "John"]},
    index=[20, 10, 30]
)

result = df.sort_index()

print(result)
```

Output

```
    Name
10  Riya
20  Amit
30  John
```

Explanation: sort_index() sorts the DataFrame using its index labels.

Easy interview explanation: Use sort_index() when you want to arrange the index.

6. Sorting by Index

Code

```
result = df.sort_index(ascending=False)

print(result)
```

Output

```
    Name
30  John
20  Amit
10  Riya
```

Explanation: ascending=False sorts index labels from largest to smallest.

Easy interview explanation: Index sorting is different from sorting by a column.

7. Missing Values While Sorting

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John", "Neha"],
    "Salary": [30000, None, 45000, 25000]
})

result = df.sort_values("Salary")

print(result)
```

Output

```
   Name   Salary
3  Neha  25000.0
0  Amit  30000.0
2  John  45000.0
1  Riya      NaN
```

Explanation: By default, missing values are placed at the end when sorting.

Easy interview explanation: Use na_position='first' to put missing values first.

8. Missing Values — NaN First

Code

```
result = df.sort_values(
    "Salary",
    na_position="first"
)

print(result)
```

Output

```
   Name   Salary
1  Riya      NaN
3  Neha  25000.0
0  Amit  30000.0
2  John  45000.0
```

Explanation: na_position='first' puts missing values before non-missing values.

Easy interview explanation: na_position can be 'first' or 'last'.

9. Stable Sorting

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John", "Neha"],
    "Score": [90, 90, 80, 90],
    "Order": [1, 2, 3, 4]
```

```
})

result = df.sort_values(
    "Score",
    kind="stable"
)

print(result)
```

Output

```
   Name  Score  Order
2  John     80      3
0  Amit     90      1
1  Riya     90      2
3  Neha     90      4
```

Explanation: A stable sort preserves the original relative order of rows with equal values.

Easy interview explanation: Stable sorting is useful when the order of tied records must remain consistent.

10. Complete Sorting Example

Code

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John", "Neha"],
    "Age": [22, 30, 25, 20],
    "Salary": [30000, 60000, 45000, 25000]
})

print("Original:")
print(df)

print("\nHighest Salary First:")
print(df.sort_values("Salary", ascending=False))

print("\nLowest Age First:")
print(df.sort_values("Age"))
```

Output

```
Original:
   Name  Age  Salary
0  Amit   22   30000
1  Riya   30   60000
2  John   25   45000
3  Neha   20   25000

Highest Salary First:
   Name  Age  Salary
1  Riya   30   60000
2  John   25   45000
0  Amit   22   30000
3  Neha   20   25000

Lowest Age First:
   Name  Age  Salary
3  Neha   20   25000
0  Amit   22   30000
2  John   25   45000
1  Riya   30   60000
```

Explanation: Sorting is useful for finding highest salary, lowest age, top scores, and ordered records.

Easy interview explanation: sort_values() is one of the most frequently used DataFrame operations.

LEVEL 10 QUICK REVISION

```
sort_values("Age")       -> Sort by column
ascending=True          -> Smallest to largest
ascending=False         -> Largest to smallest
sort_values(["A","B"])  -> Sort multiple columns
sort_index()            -> Sort by index
na_position="first"     -> Missing values first
na_position="last"      -> Missing values last
kind="stable"            -> Preserve order of equal values
```

Important interview point: sort_values() sorts by column values, while sort_index() sorts by index labels.